GPU Project 2: Optimizing SDH Computation on GPUs

Instructor: Dr. Yicheng Tu
Course: COP6527/4520 Massively Parallel Computing
Deadline: October 25, 2026 at 11:59 PM

1 Overview

In this project, you will continue working on the Spatial Distance Histogram (SDH) problem from Project 1. The main objective of Project 2 is to develop a highly efficient CUDA implementation and study the performance of the GPU memory system and other GPU optimization techniques.

In Project 1, you implemented a global-memory histogram version and a block-local shared-memory histogram version. For Project 2, performance optimization is the main focus. You should go beyond the basic shared-memory approach from Project 1 and explore techniques that improve the complete SDH computation. Unlike Project 1, your solution must also handle cases where the histogram does not fit entirely in per-block shared memory.

You may use any valid CUDA optimization techniques discussed in class or explored independently. These include, but are not limited to: tiling/caching input point data in shared memory, improving data layout and coalesced memory access, reducing thread divergence, reducing atomic contention, using private or hierarchical copies of output data, warp-level primitives and shuffle instructions, and improving workload distribution. You may combine multiple techniques.

Your code will be evaluated using multiple datasets and problem sizes. Correctness is required. All submitted code will be graded based on your rank in a class-wise contest. So do your best to optimize the code!

The accompanying paper, *Efficient 2-Body Statistics Computation on GPUs: Parallelization & Beyond*, describes several techniques relevant to this project. You are encouraged to study the paper for optimization ideas.

2 Tasks to Perform

Write a CUDA program that computes the same SDH functionality as Project 1, but with a focus on GPU performance. Perform experiments with your implementation and write a short report describing your optimization techniques and results.

Input/Output of Your Program: Your program should take care of bad or missing inputs and report appropriate errors in a reasonable way. It should be launched as follows:

```
/apps/GPU_course/runScript.sh proj2.cu {#of_samples} {bucket_width} {block_size}
```

The first two arguments are the same as in Project 1. The last argument specifies the number of threads per CUDA block. Your program must use the supplied block size rather than assuming a fixed value.

The output of your program should contain only: (1) the CPU histogram, (2) the optimized GPU histogram, (3) any differences between the two histograms, and (4) the total kernel execution time of the optimized GPU implementation.

```
******** Total Kernel Execution Time = 2.0043 sec *******
```

Project Report: Write a short report explaining how you implemented the GPU kernel(s), with emphasis on the techniques used to optimize performance. Include relevant experiments and performance results.

3 Measuring Kernel Execution Time

Use CUDA events to measure GPU kernel execution time. CUDA kernel launches are asynchronous, so the stop event must be synchronized before reading the elapsed time. A basic example is shown below.

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start, 0);
/* Your kernel call(s) go here */
cudaEventRecord(stop, 0);
cudaEventSynchronize(stop);

float elapsedTime;
cudaEventElapsedTime(&elapsedTime, start, stop);

cudaEventDestroy(start);
cudaEventDestroy(stop);
```

If your SDH computation uses multiple kernels, the reported value should cover all kernels required to compute the final GPU SDH. Do not include CPU SDH computation, GPU memory allocation, or host-device data transfers in the reported kernel execution time.

The content about measuring GPU time is based on Jason Sanders and Edward Kandrot, *CUDA by Example: An Introduction to General-Purpose GPU Programming*, Addison-Wesley, 2011.

4 Environment

All projects will be tested on CUDA 11.4 on one machine of the GAIVI cluster. If you prefer to work on your own computer, make sure your final project can be compiled and executed in the GAIVI course environment.

5 Benchmarking and Reproducibility

To make the generated input data consistent across submissions, use the following fixed random seed in the atom-list generation code:

```
srand(1);
```

This ensures that everyone uses the same generated data points and makes performance testing and grading reproducible.

For reference, the following approximate timings were provided with the Fall 2025 version of Project 2. Exact timings may differ with the current hardware and implementation.

	10,000 points	**100,000 points**	**500,000 points**
CPU	1.2 s	129 s	1 hour
GPU Project 1 (2025)	25 ms	1880 ms	45 s
GPU Project 2 (2025)	8.6 ms	500 ms	11 s

6 Instructions to Submit Project

Submit one .cu file containing your implementation of the SDH algorithm and one separate .pdf file containing your report. Put both files in a folder, compress the folder, and name the compressed file proj2-xxx.zip (or proj2-xxx.tar), where xxx is your USF NetID. Use your NetID, not your UID.

Submit the compressed file only through the Project 2 assignment link in Canvas. E-mail or other forms of submission will not be graded. After submitting, download your submission from Canvas and verify that it opens

correctly.

7 Rules for Grading

The functionality of the project will be graded using a set of test cases. Keep the following rules in mind:

- The SDH computed by your CUDA code must be exactly the same as the SDH computed by the CPU code.
- The output should contain only the CPU histogram, the optimized GPU histogram, any differences between the two histograms, and the total kernel execution time.
- All programs that fail to compile will receive zero points. Your submission must compile using the course run script.
- Example: /apps/GPU_course/runScript.sh proj2.cu 100000 500 256
- Your program must correctly support different numbers of samples, bucket widths, and CUDA block sizes supplied through the command line.
- Your implementation must respect GPU resource limits, including shared-memory capacity, and must not assume that the histogram always fits entirely in shared memory.
- If changes are required after the submission deadline to make the main code work, they must be limited to at most three lines of code and will incur a 30% penalty.
- Do not use tricks that effectively run the computation on one thread or on the CPU.
- Performance-measurement code will be checked carefully. Manipulating timing to report a shorter value than the actual kernel execution time will be treated as cheating and incur a heavy penalty.